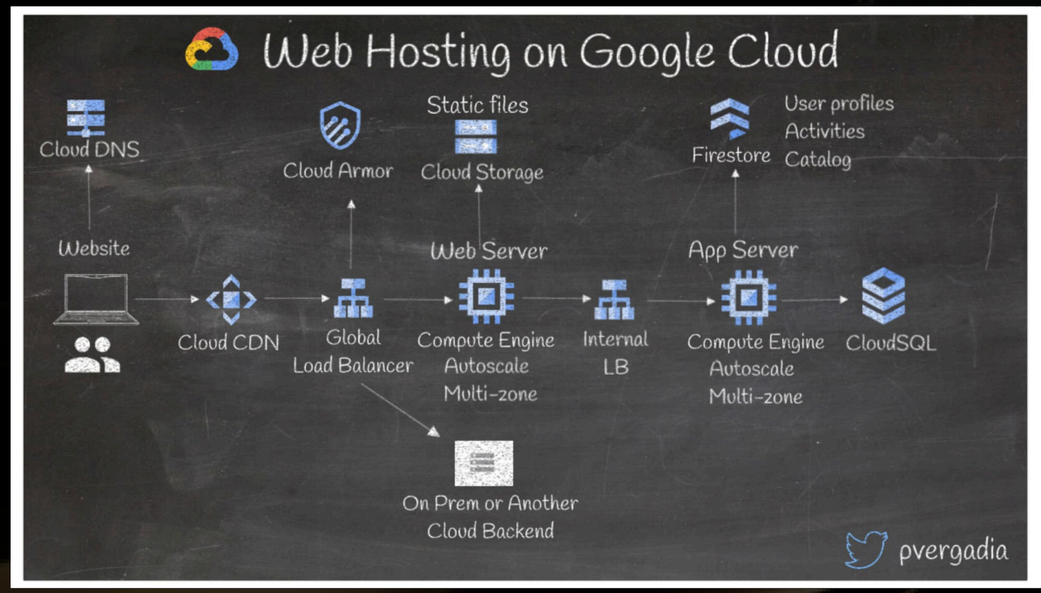
Cloud Architecture Diagram Generator

주환석

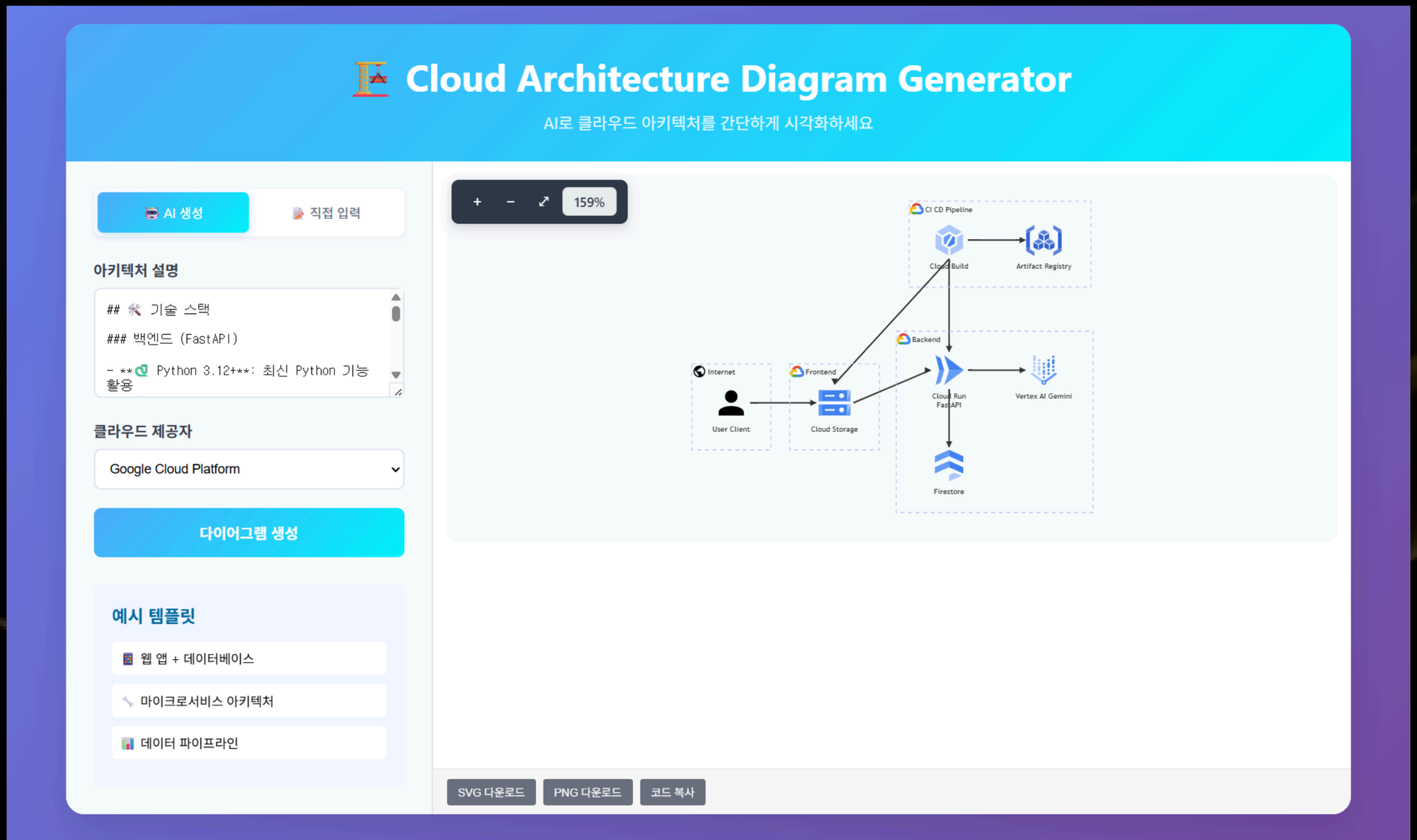
GemiNight (Gemini 실습하는 개발자들의 밤)

프로젝트 개요

- 클라우드 도식화에 관심이 많음
  - 이것저것 써봄 / 수동 / 자유도 ▼ / 유료...
- AI 기반 클라우드 아키텍처 다이어그램 자동 생성 웹 애플리케이션
- 자연어 설명을 다이어그램으로 변환



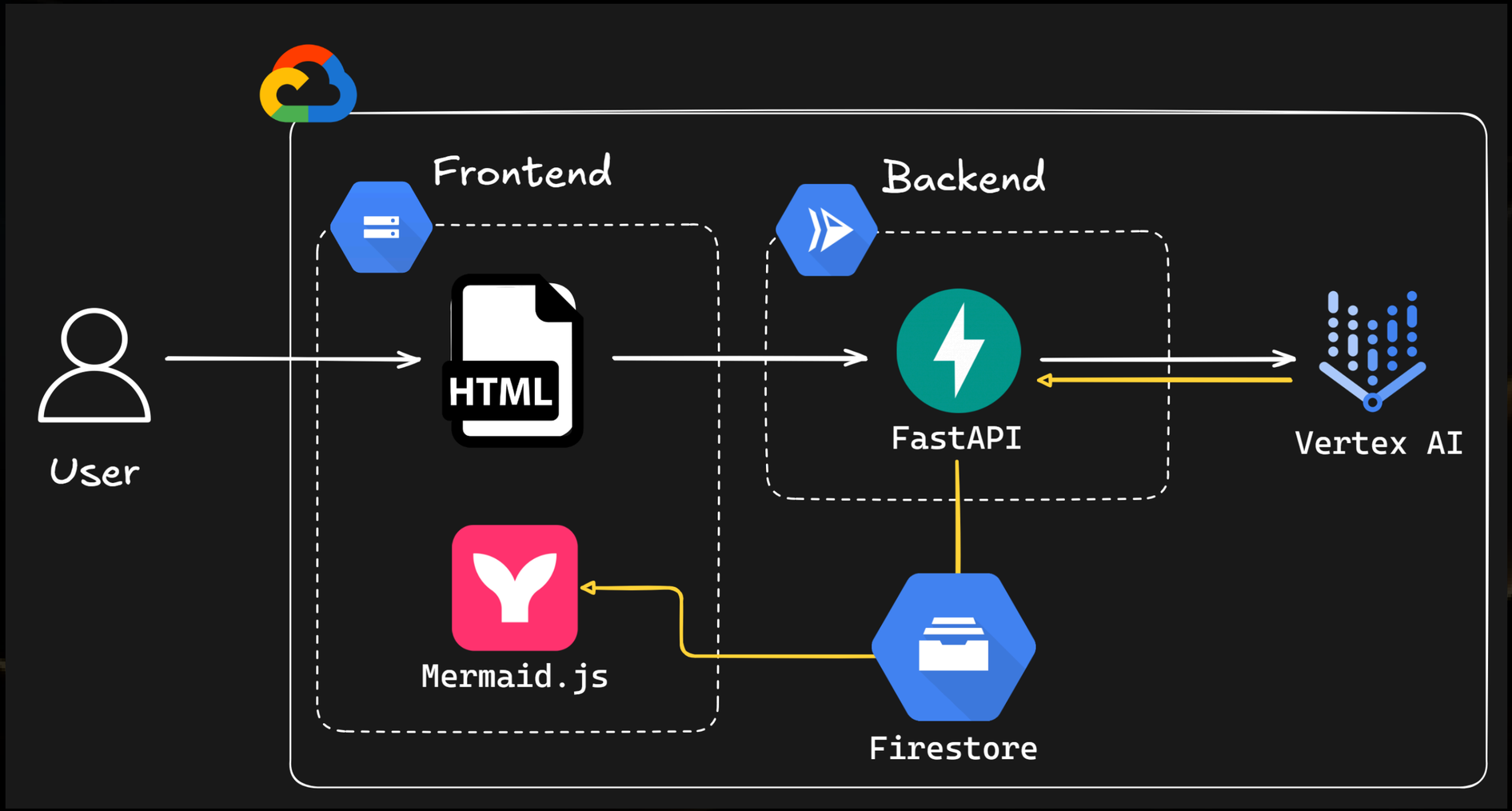
그래서 만들어 봤음


Cloud Architecture Diagram Generator
AI로 클라우드 아키텍처를 간단하게 시각화하세요
AI 생성
직접 입력
아키텍처 설명
기술 스택
백엔드 (FastAPI)
- **Python 3.12+**: 최신 Python 기능 활용
클라우드 제공자
Google Cloud Platform
다이어그램 생성
예시 템플릿
웹 앱 + 데이터베이스
마이크로서비스 아키텍처
데이터 파이프라인
159%
CI CD Pipeline
Cloud Build
Artifact Registry
Backend
Internet
User Client
Frontend
Cloud Storage
Cloud Run
FastAPI
Vertex AI Gemini
Firestore
SVG 다운로드
PNG 다운로드
코드 복사

기술 스택

- 🤖 Google Vertex AI (Gemini 2.5-flash): AI 모델
- ☁️ Google Cloud Run: 서버리스 컨테이너 플랫폼 (백엔드)
- ☁️ Google Cloud Storage: 정적 웹사이트 호스팅 (프론트엔드)
- ☁️ Google Cloud Build + Artifact Registry (CI/CD)
- ✨ Mermaid.js: architecture-beta (다이어그램 랜더링)

아키텍처


Frontend
Backend
User
HTML
FastAPI
Vertex AI
Mermaid.js
Firestore

주요 기능

- 🤖 AI 기반 다이어그램 생성: Google Gemini를 사용한 자동 아키텍처 다이어그램 생성
- ☁️ 다중 클라우드 지원: GCP, AWS, Azure 아키텍처 지원
- 📊 실시간 시각화: Mermaid.js: architecture-beta 기반 즉시 렌더링
- 📥 다운로드 기능: SVG/PNG 형식으로 다이어그램 내보내기
- 📋 코드 복사: Mermaid 코드 클립보드 복사
- 🎨 클라우드 아이콘: 각 클라우드 제공자별 공식 아이콘 지원
- 💾 ~~다이어그램 관리: Firestore를 통한 다이어그램 저장/조회/수정/삭제~~

시연

🏗️ Cloud Architecture Diagram Generator

프로젝트 구조

```
gcp-cloud-architecture/
├── backend/                    # 🔧 FastAPI 백엔드 서비스
│   ├── main.py
│   ├── Dockerfile
│   └── cloudbuild.yaml
├── frontend/                   # 🎨 웹 프론트엔드
│   ├── index.html
│   ├── app.js
│   ├── styles.css
│   └── icons/                  # 클라우드 아이콘
│       ├── gcp.json
│       ├── aws.json
│       └── azr.json
```

Vertex AI

- gemini-2.5-flash
- 유저 입력 + 탬플릿 프롬프트
- Mermaid.js: architecture-beta 형태 문자열 생성

prompt

```
prompt = """{ 유저 입력 }
            - (... 이렇게 해줘 /  저렇게 해줘 ...)
            - 다음 설명을 바탕으로 클라우드 아키텍처 다이어그램을  Mermaid.js: architecture-beta 코드로 생성"""
```

모델 호출

```
model = GenerativeModel("gemini-2.5-flash")
response = model.generate_content(prompt)
```

Javascript

response로 받은 문자열

- Mermaid.js: architecture-beta 이용하여 이미지로 랜더링

```
architecture-beta
    group vpc(gcp:vpc)[Virtual Private Cloud]
    service loadbalancer(gcp:load-balancing)[Load Balancer] in vpc
    service cloudrun(gcp:cloud-run)[Cloud Run] in vpc
    service cloudsql(gcp:cloud-sql)[Cloud SQL] in vpc
```

커스텀 아이콘 등록 필요

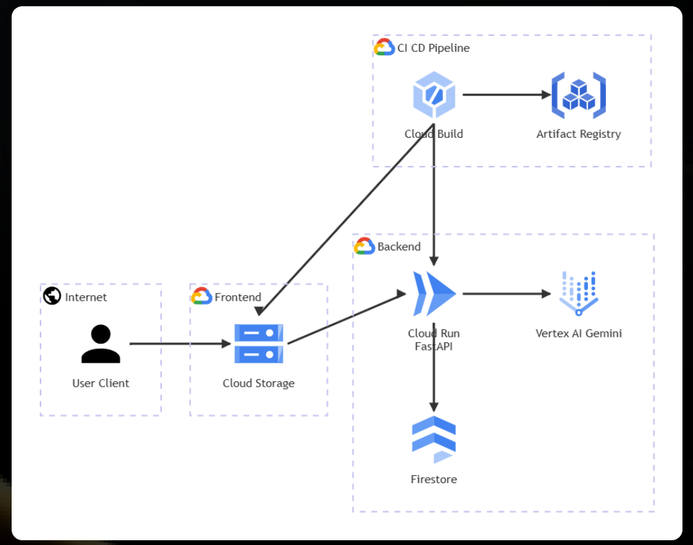
```
// Register icon packs
mermaid.registerIconPacks([
    { name: 'gcp', loader: () => fetch('./icons/gcp.json').then(res => res.json()) },
]);
```

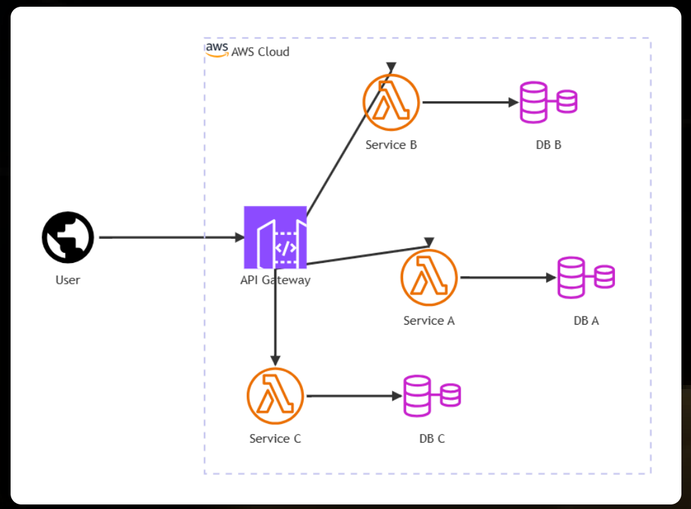
클라우드 아이콘 JSON

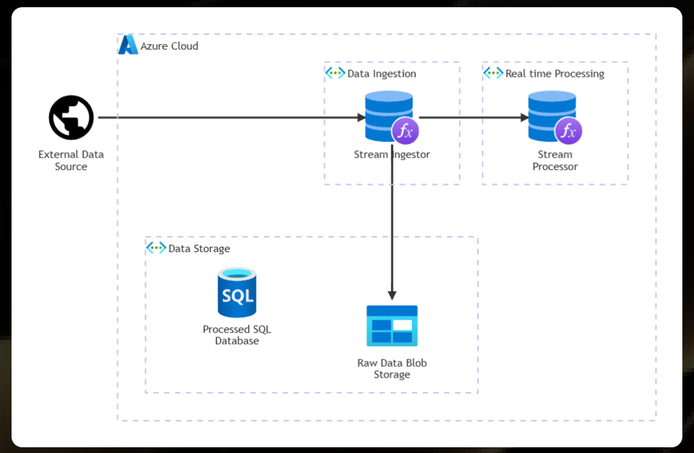
- prefix
- icons

```
{
    "prefix": "gcp",
    "icons": {
        "cloud_monitoring": {
            "width": 24,
            "height": 24,
            "body": "<style>.cls-1{fill:#669df6;}.cls-2{fill:#4285f4;}</style>" //svg 파일
    }
}
```

프로젝트 결과


CI CD Pipeline
Cloud Build
Artifact Registry
Backend
Internet
Frontend
User Client
Cloud Storage
Cloud Run
FastAPI
Vertex AI Gemini
Firestore


AWS Cloud
Service B
DB B
User
API Gateway
Service A
DB A
Service C
DB C


Azure Cloud
Data Ingestion
Real time Processing
External Data
Source
Stream Ingestor
Stream
Processor
Data Storage
SQL
Processed SQL
Database
Raw Data Blob
Storage

트러블 슈팅

Cloud Build

- 불필요한 파일이 함께 업로드 되는 문제
- ❗ .gcloudignore 파일

Cloud Storage

- 배포 후 캐싱으로 인해, 제대로 테스트하기 어려움
- ❗ 캐싱 무효화

```
NO_CACHE="Cache-Control:no-cache, no-store, must-revalidate, max-age=0"

gsutil -h "$NO_CACHE" cp ...
```

개선 사항

- 프롬프트 개선
  - Mermaid 문법 에러 → ❓템플릿 프롬프트
  - 매번 긴 템플릿 프롬프트 → ❓어떻게 학습 시켜둘수 있나 = 튜닝
  - 정확도 개선 (서비스 이름 매칭) → ❓RAG or MCP
  - 속도 개선 → ❓고성능 모델
- API 기능 추가
  - Firestore를 통한 다이어그램 저장/조회/수정/삭제

한계점

- 매번 다른 결과
- 복잡한 클라우드 아키텍처는 무리일지도...
- Mermaid.js: architecture-beta 아직까지는 아쉬움이 많음
  - 연결 선
  - 도형 배치 레이아웃